

Mentoria de IA aplicada · **Aula 6 de 8**

Orquestração com LangGraph.js

O fluxo vira um grafo: estado, caminhos por confiança e "salvar jogo"

Hoje

Ao final, você consegue:

- modelar um fluxo com **estado, nós e arestas**
- usar **arestas condicionais** e **reducers**
- explicar **checkpoint, thread** e retomada
- dizer o que a **viagem no tempo** resolve
- escolher entre **LangGraph** e as alternativas

Roteiro (40 min)

1. As peças de um grafo
2. Demo: o curador como grafo
3. Checkpoint e retomada
4. Alternativas

Depois: 15 min de discussão

Um workflow vira grafo quando você precisa **retomar, desviar ou pausar**.

Se o fluxo é curto e reto, uma função com `if` basta.

O que faltava no time da Aula 5

Problema	No pipeline em código	No grafo
Caiu no último passo	refaz tudo (~14 min)	retoma do checkpoint
Confiança baixa	segue o mesmo caminho	desvia para a fila humana
Edição sem regulação nem mercado	"top 10 por confiança"	cota por categoria no nó de seleção
Entender o fluxo	ler o código	desenhar o grafo

1 • As peças de um grafo

Estado, nós e arestas

Estado

A **pasta** que passa de mesa em mesa: campos tipados (Zod)

Nó

Uma **função**: recebe o estado e devolve **só o que mudou**

Aresta

Depois deste, vá para aquele. A condicional escolhe pelo estado

```
new StateGraph(EstadoTriagem)
  .addNode("classificar", async (e) => await classificar(e.chamado))
  .addNode("financeiro", atender("financeiro")) // ... suporte, faq, humano
  .addEdge(START, "classificar")
  .addConditionalEdges("classificar", rotaDeTriagem, ["financeiro", "suporte", "faq", "humano"])
```

Reducers: como juntar atualizações

Um nó devolve `{ avaliados: [x] }`.
E o que já estava lá?

- **Sem reducer:** o novo **substitui**
- **Com reducer:** o LangGraph **combina** (aqui, concatena)

Essencial quando nós **em paralelo** escrevem no mesmo campo.

Fan-out com Send

```
candidatos.map(  
  (id) => new Send("classificar", { id })  
)
```

Uma execução do nó por item, em paralelo.
Todas escrevem em `avaliados`.

Duas armadilhas que pegamos

Condicional logo após fan-out

Roda **uma vez por ramo**: 32 ramos, 32 decisões, itens repetidos.

Solução: um **nó de junção** (selecionar) com aresta normal. Ele roda uma vez, com tudo pronto.

Nó com nome de campo

"atendidoPor" is already being used as a state attribute

Solução: nomes diferentes para nós e campos.

2 · O curador como grafo

demo:grafo

O grafo do curador

```
START → coletar -(Send × N)→ classificar → selecionar —(Send)→ redigir ≠ revisor —  
      | sem candidatos → END                                     | (Send)→ fila_humana —————→ publicar → END  
      | nada a fazer —————
```

- **classificar:** um por notícia, em paralelo
- **selecionar:** confiança $\geq 0,9$, até 10 itens, **no máximo 3 por categoria**
- **fila_humana:** confiança baixa ou guardrail. Na Aula 7, vira uma **pausa** de verdade
- Os papéis da Aula 5 entram **sem mudar uma linha**

Resultado do grafo

Etapa	Chamadas	Resultado
coletar	1 agente	32 candidatos
classificar (paralelo)	32	32 de 32 iguais ao gabarito
selecionar	0 (código)	10 escolhidos · 4 para a fila humana · 15 fora pela cota/limite
redigir $\rightleftharpoons$ revisor	16 + 16	9 aprovados · n10 reprovada 2x

- **Regulação voltou** (3 itens) graças à cota. **Mercado continua fora:** o modelo dá **0,95** a todo item de mercado e **0,98** aos outros

3 · Checkpoint e retomada

demo:checkpoint

"Salvar jogo" a cada passo

```
.compile({ checkpointer })  
await grafo.invoke(entrada,  
  { configurable: { thread_id: "edicao-da-semana" } })
```

- **Thread:** uma execução (uma conversa, uma edição)
- **Retomar:** `invoke(null, config)`
- **Memória:** mesmo `thread_id`, chamada seguinte
- **Inspecionar:** `getState`, `getStateHistory`

Onde guardar

MemorySaver : testes

Arquivo JSON: para ver que é **só dado**

Banco (SQLite, Postgres): produção

A retomada no curador

1ª execução · --falhar

coletar 1 · classificar 32 · redigir 16 · revisar 16

663 s · parou em publicar

2ª execução · retomada

coletar 0 · classificar 0 · redigir 0 · revisar 0

1 s · só publicar rodou

Viagem no tempo

O classificador mandou "Não recebi o reembolso" para o **FAQ**. Era **cobrança**.

1. Ache no histórico o checkpoint depois de `classificar`
2. `updateState` grava **cobrança** ali, como se `classificar` tivesse respondido isso
3. `invoke(null, ...)` segue do **ramo** novo: só o atendimento roda de novo

O passado fica

O ramo errado **continua** no histórico. Serve para auditar e para criar exemplos de avaliação (Aula 3).

4 • Alternativas

Quando usar o quê

Não é só LangGraph

Opção	Em uma frase	Escolha quando
Código puro	funções, laços e <code>if</code>	fluxo curto, sem retomar nem pausar
Vercel AI SDK	chamadas, ferramentas e loop de um agente	o agente é o produto
LangGraph.js	grafo com estado e checkpoint	fluxos longos, com ramos, falhas e humanos
Mastra	workflows em TS (<code>.then</code> , <code>.branch</code> , <code>.parallel</code>) + suspender/retomar	framework completo, API encadeada
OpenAI Agents SDK	agentes com handoffs, guardrails e aprovação	atendimento com transferência entre especialistas
Claude Agent SDK	o motor do Claude Code como biblioteca	agente que trabalha com arquivos e comandos

Como escolher

1. Precisa **retomar, pausar ou voltar no tempo**? → algo com checkpoint
2. O fluxo é **fixo** ou o **modelo decide** os passos? → workflow ou agente (Aula 4)
3. Quanta **abstração** a equipe aguenta?
4. **Prenda-se pouco**: papéis como funções simples; o framework só **liga**

Discussão

15 minutos

Para conversar

1. O curador precisava de um grafo, ou o pipeline da Aula 5 bastava? O que pesou?
2. Que processo do seu trabalho **cai no meio** e hoje recomeça do zero?
3. Onde um processo precisa **lembrar** de interações anteriores? O que seria a thread?
4. A cota de 3 por categoria é uma regra editorial no **código**. Quem deveria decidir essa regra?
5. Para que serve guardar o caminho **errado** no histórico?

Para a próxima aula

Exercício

Grafo de triagem de chamados

```
pnpm -F @mentoria/ex06-langgraph test
```

Desafio: viagem no tempo

Projeto final · M6

O curador como grafo, com checkpoint

```
pnpm -F @mentoria/curador m06:grafo
```

Traga o relatório e a fila humana

Próxima aula · **Human-in-the-loop**

**E se a fila humana virasse uma
pausa, e o grafo esperasse alguém
aprovar?**

`interrupt()`, `Command({ resume })` e o que automatizar 100%.